Example: Test Drivers – Test Subs

Consider the following hierarchy of classes for shapes:

```
Shape
 |
 +-- Rectangle
 |
 +-- Circle
```

With the two classes Rectangle and Circle implemented as

```python
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def circumference(self):
        return 2 * math.pi * self.radius
```

We can create a `TestDriver` class that tests the behavior of the `Rectangle` and `Circle` classes. The `TestDriver` class can create instances of `Rectangle` and `Circle`, call their methods, and verify that they produce the expected results.

Here is an example:

```
class TestDriver:
    def run_tests(self):
        # Test Rectangle class
        rectangle = Rectangle(10, 20)
        assert rectangle.area() == 200
        assert rectangle.perimeter() == 60

        # Test Circle class
        circle = Circle(5)
        assert circle.area() == 78.5
        assert circle.circumference() == 31.4
```

In this example, the `TestDriver` class creates an instance of the `Rectangle` class and tests its `area()` and `perimeter()` methods, verifying that they return the expected results. Similarly, it creates an instance of the `Circle` class and tests its `area()` and `circumference()` methods.

Now, suppose that the `Rectangle` class depends on an external resource, such as a database, to retrieve its dimensions. In order to test the behavior of the `Rectangle` class in isolation, we can create a `TestStub` class that simulates the behavior of the external resource. Here's an example:

```
class RectangleTestStub:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def get_dimensions(self):
        return (self.length, self.width)


class TestDriver:
    def run_tests(self):
```

```
    # Test Rectangle class with test stub
    rectangle_stub = RectangleTestStub(10, 20)
    rectangle = Rectangle(rectangle_stub.get_dimensions())
    assert rectangle.area() == 200
    assert rectangle.perimeter() == 60

    # Test Circle class
    circle = Circle(5)
    assert circle.area() == 78.5
    assert circle.circumference() == 31.4
```

In this example, the `RectangleTestStub` class simulates the behavior of the external resource by returning a fixed set of dimensions. The `TestDriver` class creates an instance of the `RectangleTestStub` class, passes it to the `Rectangle` class, and tests its behavior as before.

Note that in this example, the `Rectangle` class now takes a `RectangleTestStub` object as input, instead of the actual length and width of the rectangle. This is because the `Rectangle` class now relies on the `get_dimensions()` method of the `RectangleTestStub` object to retrieve its dimensions.

In summary, the use of test stubs and test drivers allows us to test the behavior of software components in isolation, without being affected by the behavior of external resources or dependencies.